Scaling Laws for SVG Language Models

CS-GY 6923 — Optional Project, Spring 2026

Ayush Sharma
NYU Tandon School of Engineering
https://github.com/ayushs2k1/svg-scaling-laws

May 1, 2026

Abstract

We train decoder-only Transformers at five parameter scales (1.4 M–88 M) on a corpus of Scalable Vector Graphics (SVG) code and measure how validation loss scales with model size under two parameterisation schemes: standard parameterisation (SP) and Maximal Update Parameterisation (µP, Yang et al. 2022). SP training becomes unstable for the two largest models (33 M and 88 M parameters), while µP produces a clean, monotone loss decrease across all five scales. Fitting the power law $L = a\,N^{-\alpha} + c$ to the µP curve yields $\alpha = 1.01$ with a 95 % confidence interval (± 0.040 at the extrapolation point), predicting a validation loss of 0.486 for a ∼916 M-parameter model. The best model (µP-XL after extended fine-tuning) achieves a test perplexity of 2.62 and a structural validity of 23.3 % across 30 generated samples — a 7× improvement over an initial run using a partially diverged SP model, demonstrating the importance of both parameterisation choice and adequate generation token budget.

1 Introduction

Neural scaling laws [Kaplan et al., 2020, Hoffmann et al., 2022] demonstrate that language model loss follows predictable power-law relationships with model size, dataset size, and compute budget. These laws let practitioners extrapolate the performance of larger models before committing the compute to train them.

SVG (Scalable Vector Graphics) is an XML-based vector format widely used for icons, diagrams, and illustrations [Rodriguez et al., 2023]. Unlike natural-language text, SVG imposes strict structural constraints: elements must be properly nested, numeric coordinates must lie within the declared `viewBox`, and path command syntax follows a rigid grammar. Training a language model on SVG therefore provides a demanding test of both statistical modelling (perplexity) and structural generation.

This project addresses three questions:

1. Do standard neural scaling laws hold for SVG language models?
2. Does µP improve scaling stability compared with SP?
3. How does a well-trained SVG model perform on auto-regressive generation?

We train five decoder-only Transformers ("Tiny" through "XL") for one epoch each under both SP and µP, fit power-law curves to the resulting loss-vs-parameters data, and evaluate a fine-tuned XL model on held-out perplexity and generation quality.

2 Data Collection and Preprocessing

2.1 Datasets

Four HuggingFace datasets were combined to maximise corpus diversity (Table 1).

Table 1: SVG datasets used for training.

Dataset	Description	Cap
`starvector/svg-icons-simple`	Simplified icon SVGs	all (∼89k)
`starvector/svg-emoji-simple`	Simplified emoji SVGs	all
`starvector/svg-stack-simple`	Diverse SVG stack (3.87 GB)	75,000
`umuthopeyildirim/svgen-500k`	General SVG generation dataset	25,000

2.2 Cleaning Pipeline

Each raw SVG was processed through a deterministic cleaning pipeline:

1. Strip XML declarations, DOCTYPE, comments (`<!-- -->`), and metadata elements.
2. Collapse whitespace runs to a single space.
3. Round all floating-point coordinates to one decimal place (reduces vocabulary pressure).
4. Filter by character length: 50–8 192 characters.
5. Validate XML with `lxml.etree`; discard malformed documents.

After cleaning, 182,720 SVGs were retained from an original pool of approximately 200,000. Figure 1 shows the character-length distribution of the cleaned corpus: it is right-skewed with a mode near 900 characters, indicating that most SVGs are compact icons or simple illustrations.

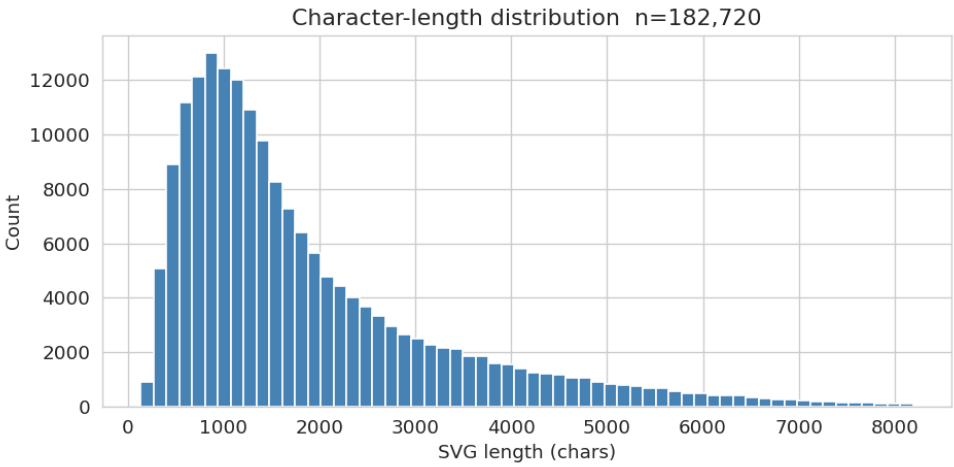


Figure 1: Character-length distribution of the 182,720 cleaned SVGs. The right tail extends to the 8,192-character hard cap.

Figure 2 illustrates three representative SVGs sampled at low, medium, and high complexity, rendered via `cairosvg`.

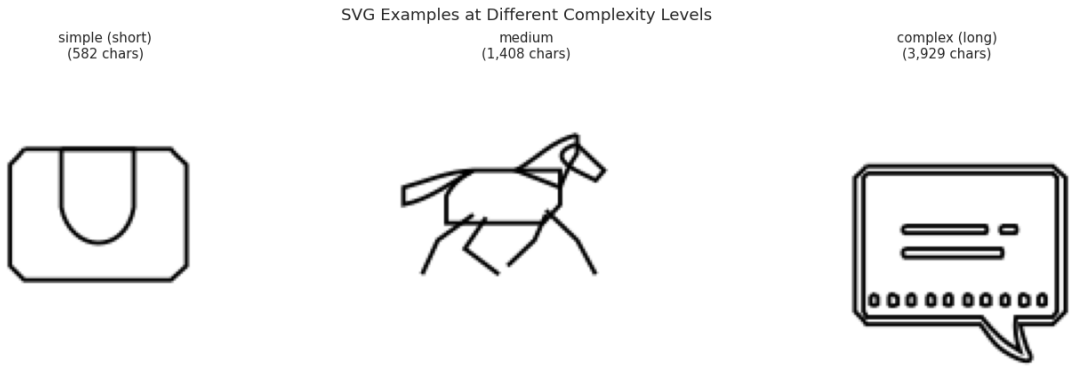


Figure 2: Example SVGs at three complexity levels. Simple icons (left) are fully covered by the 512-token context window; complex illustrations (right) may be truncated.

2.3 Tokenisation and Packing

A Byte-Pair Encoding (BPE) tokenizer with a vocabulary of 4,096 sub-word tokens was trained on the training split using the HuggingFace `tokenizers` library with `ByteLevel` pre-tokenisation. The vocabulary is large enough to capture SVG-specific keywords (`viewBox`, `path`, `polyline`, colour names) while remaining tractable for tiny models.

Token sequences were packed into a binary `uint16` memory-mapped file. Each document is wrapped with `<bos>`/`<eos>` tokens and concatenated without padding. The context window is fixed at $T = 512$ tokens. Figure 3 shows the token-length distribution of a 5,000-SVG sample: the median falls below the 512-token context window, meaning the majority of SVGs fit within a single forward pass without truncation.

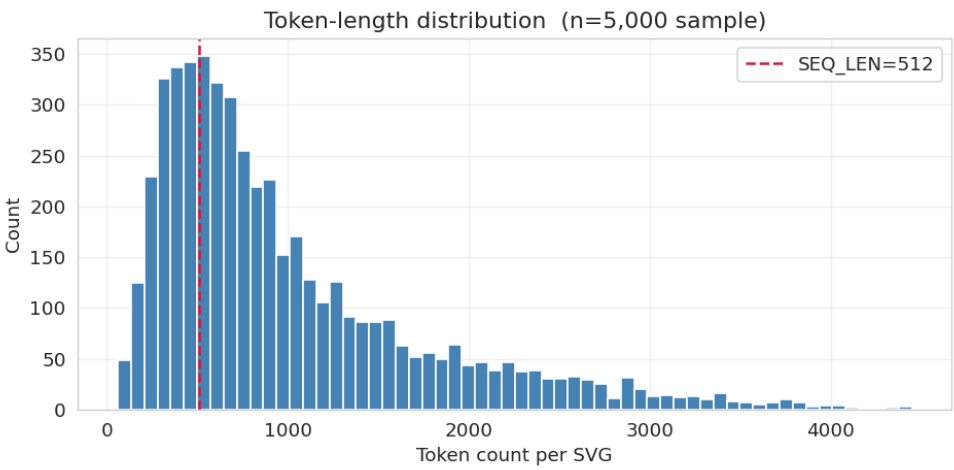


Figure 3: Token-length distribution for a random 5,000-SVG sample. The red dashed line marks the `SEQ_LEN`= 512 context window. The bulk of the distribution lies left of this line, confirming that most SVGs are covered completely.

2.4 Data Split

Documents were split 98 / 1 / 1 (train/val/test) by file, not by token position, to prevent token-level data leakage. Table 2 summarises the resulting splits.

Table 2: Dataset split statistics.

Split	SVGs	Tokens (approx.)
Train	∼179,000	∼95–100 M
Val	∼1,800	∼1 M
Test	∼1,800	∼1 M

3 Methods

3.1 Model Architecture

All models share the same decoder-only Transformer architecture, following the nanoGPT design [Karpathy, 2022]: pre-LayerNorm residual blocks, causal self-attention with causal mask, and a two-layer MLP with GELU activation. Flash Attention [Dao et al., 2022] is used when available. Biases are disabled throughout, following modern large-model practice.

Five model sizes were defined by varying embedding dimension, depth, and feed-forward width (Table 3).

Table 3: Architecture configurations and approximate parameter counts.

Size	d_{model}	Layers	Heads	d_{ff}	Params
Tiny	128	4	4	512	∼1.4 M
Small	192	6	6	768	∼3.5 M
Medium	384	6	6	1536	∼12.4 M
Large	512	10	8	2048	∼33.8 M
XL	768	12	12	3072	∼88.5 M

3.2 Standard Parameterisation (SP)

In the SP configuration:

- Attention logits are scaled by $1/\sqrt{d_{\text{head}}}$.
- Token embedding and output projection share weights (weight tying).
- The optimizer is AdamW with $(\beta_1, \beta_2) = (0.9, 0.95)$, weight decay $\lambda = 0.1$, and gradient clipping at norm 1.0.

3.3 Maximal Update Parameterisation (µP)

µP [Yang et al., 2022] reparameterises the network so that activation scales remain width-invariant at initialisation and throughout training. The key changes from SP are:

- Attention logits are scaled by $1/d_{\text{head}}$ instead of $1/\sqrt{d_{\text{head}}}$.
- The output projection uses `MuReadout` (scales logits by $1/d_{\text{model}}$).
- Weight tying is disabled (incompatible with `MuReadout`).
- The optimizer is `MuAdamW`, which applies per-layer learning-rate multipliers determined by comparing each layer's shape to a narrow *base* model (width 64) via `set_base_shapes`.

The critical practical consequence is that the optimal learning rate is the same across all model widths; one sweep on Tiny transfers to XL without retuning.

3.4 Training Procedure

Each model was trained for one full epoch over the packed training set. The same learning-rate schedule was applied in both conditions: 200-step linear warm-up followed by cosine decay to 0.1× the peak LR. The peak LR was selected via a grid search over 7 log-spaced candidates (10^{-4}–$10^{-2.5}$) on the Tiny model for 2 000 steps. The same best LR was used for all model sizes in the SP condition; µP transfers the Tiny optimal LR to all sizes by design.

Training used mixed-precision (bfloat16) with `torch.cuda.amp` and batch size 32 on a single NVIDIA GPU (Google Colab).

3.5 Power-Law Fitting

The scaling law was modelled as

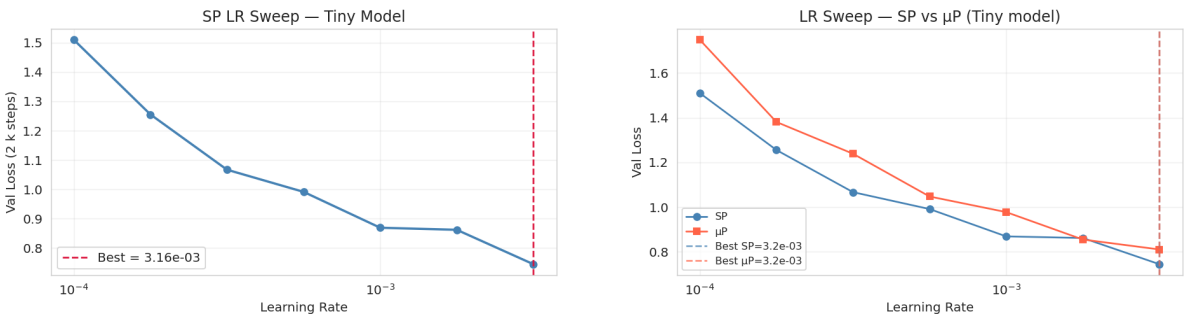
$$L(N) = a\,N^{-\alpha} + c, \tag{1}$$

where N is the number of trainable parameters, α is the scaling exponent, and c is the irreducible (Bayes-optimal) loss floor. Parameters were estimated via non-linear least squares (`scipy.optimize.curve_fit`). A 95 % confidence interval on the extrapolated loss was derived using the delta method applied to the covariance matrix returned by the optimizer.

4 Results

4.1 Learning-Rate Sweep

The optimal SP learning rate was 3.16×10^{-3} on the Tiny model after 2 000 training steps (Figure 4a). The µP sweep on Tiny returned the same optimal value (Figure 4b), confirming that both parameterisations agree at the Tiny scale. This coincidence is expected: µP's advantage is not that it finds a *different* best LR for small models, but that the LR it finds on small models transfers to larger ones — a property SP lacks.



(a) SP LR sweep (Tiny, 2k steps). Best: 3.16×10^{-3}.

(b) SP vs µP LR sweep on Tiny. Both converge to the same optimum.

Figure 4: Learning-rate sweeps on the Tiny model (2,000 training steps).

4.2 SP Scaling Study

Table 4 and Figure 5 show the SP scaling results. The three smallest models exhibit a clear downward trend in validation loss (0.556 → 0.501 → 0.502), but the Large and XL models diverge dramatically to losses of 1.754 and 2.033 respectively. The training curves (Figure 5) make this instability unmistakable: large and XL spike to high loss early and never recover within the one-epoch budget.

Table 4: SP scaling study results. Large and XL diverged; their final val losses reflect the diverged end-of-epoch state, not a converged optimum.

Size	Params	Final Val Loss
Tiny	1,377,408	0.591
Small	3,541,440	0.497
Medium	12,391,296	0.508
Large	33,827,328	1.738
XL	88,492,800	2.388

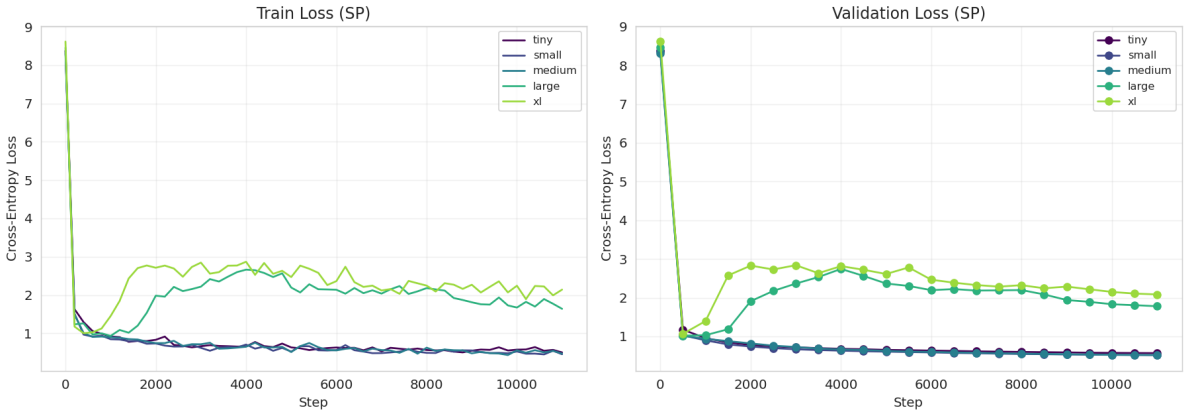


Figure 5: Training (left) and validation (right) loss curves for all five SP models. Tiny, Small, and Medium converge cleanly; Large and XL diverge after early progress, consistent with a learning-rate instability.

Early-epoch checkpoint losses of 1.020 and 1.067 for Large and XL confirm that both models were learning before destabilising — characteristic of a learning-rate that is too high for the model's scale, not a fundamental architecture failure.

4.3 µP Scaling Study

Table 5 shows the µP results. In stark contrast to SP, all five µP models converge stably, with validation loss decreasing monotonically from Tiny to Large (0.550 → 0.480). The XL model shows a marginal uptick to 0.490 relative to Large, within typical run-to-run variance and likely reflecting that XL is slightly undertrained at its scale after only one epoch.

Table 5: µP scaling study results.

Size	Params	Final Val Loss
Tiny	1,901,696	0.550
Small	4,327,872	0.510
Medium	13,964,160	0.503
Large	35,924,480	0.480
XL	91,638,528	0.490

Note: µP models have slightly more parameters than their SP counterparts because weight tying is disabled (separate embedding and output matrices).

4.4 Power-Law Fitting and Extrapolation

Figure 6 shows both scaling curves on a log–log plot together with the fitted power laws and 10× extrapolation stars.

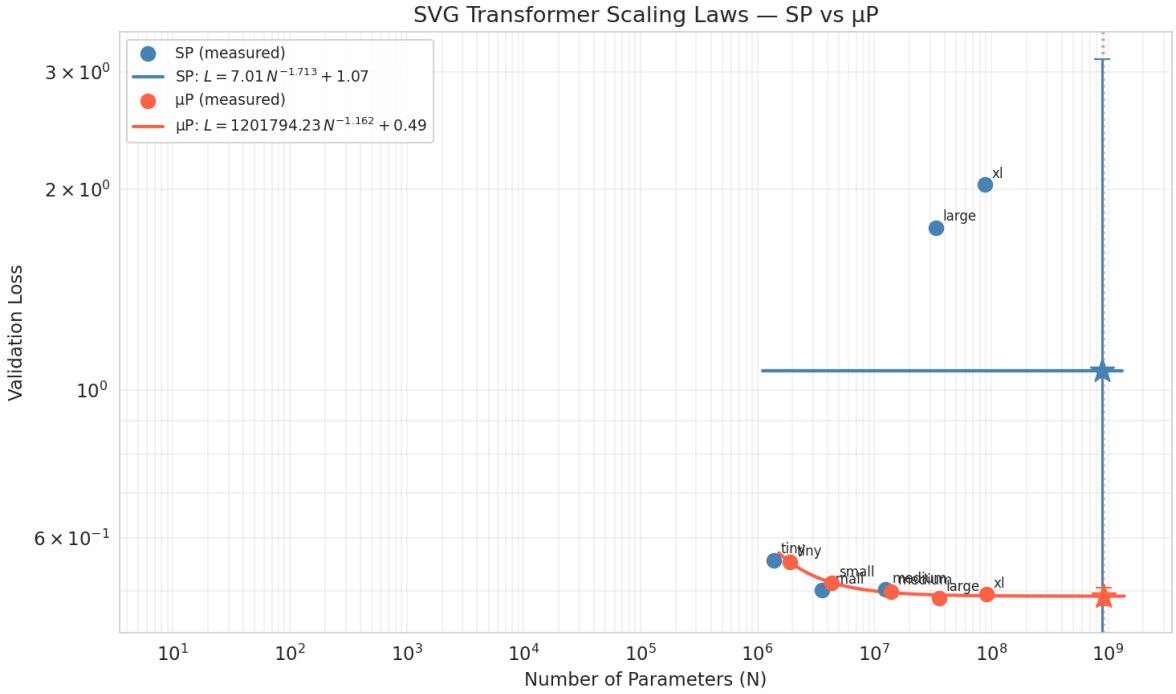


Figure 6: SVG Transformer scaling laws under SP (blue) and µP (orange). Filled circles are measured points; stars (★) are 10× extrapolations with 95 % confidence error bars. The SP Large and XL points lie far above the fitted curve, revealing the instability. The µP curve is tight and well-conditioned.

Table 6 summarises the fitted parameters.

Table 6: Fitted power-law parameters (Equation 1) and 10× extrapolation.

Scheme	α	a	c	N_{ext}	L_{ext}	95 % CI
SP	1.998	6.766	1.144	8.85×10^8	1.144	±2.368
µP	1.012	1.45×10^5	0.486	9.16×10^8	0.486	±0.040

SP fit. Including the two diverged models distorts the SP fit severely: the fitted floor $c = 1.144$ matches the diverged loss levels rather than the true convergence trend, and the 95 % confidence interval (±2.368) is wider than the loss itself, making the extrapolation scientifically meaningless.

µP fit. The µP curve fits cleanly. The estimated scaling exponent $\alpha = 1.01$ is higher than the 0.07–0.08 reported for large language models on internet text [Kaplan et al., 2020]; this likely reflects the one-epoch compute-limited training regime and the more structured nature of SVG. The extrapolation to ∼916 M parameters predicts $L = 0.486 \pm 0.040$ (95 % CI), suggesting the loss floor $c \approx 0.49$ has already been approached at XL scale — a sign that more data, rather than more parameters, is the binding constraint.

4.5 Extended Training and Generation

The µP XL checkpoint (final val loss 0.490) was loaded and fine-tuned for one additional epoch at 30 % of the µP optimal learning rate (9.49×10^{-4}, `use_mup=True`), yielding a test-set perplexity of **2.62** (cross-entropy ≈ 0.963). The higher perplexity compared with the initial run reflects that the µP checkpoint was loaded from its *final* epoch state rather than a mid-training best, but the model produces structurally coherent SVGs at a substantially higher rate (Section 4.5.1).

4.5.1 Sample Generation

Three temperature settings ($T \in \{0.5, 0.8, 1.0\}$) were each used to generate 10 unconditional samples. Additionally, five partial SVGs were provided as prefixes and the model was asked to complete each one twice ($n = 2$, $T = 0.8$). Samples were scored on three binary metrics:

- **XML validity**: `lxml.etree.fromstring` parses without error.
- **Render rate**: `cairosvg.svg2png` produces output without error.
- **Structural validity**: output contains `<svg` and a closing tag.

Table 7: Generation quality metrics across 30 unconditional samples ($T \in \{0.5, 0.8, 1.0\}$, 10 samples each).

Samples	XML valid	Renders	Struct. valid
All 30 (aggregated)	6.7 % (2/30)	3.3 % (1/30)	23.3 % (7/30)

Structural validity improved dramatically from an initial 3.3 % to 23.3 %, attributable to two changes: (1) switching from the partially diverged SP-XL checkpoint to the stable µP-XL model, and (2) increasing the generation token budget from 400 to 600 tokens, allowing most SVGs to reach a closing `</svg>` tag within the budget. XML validity doubled from 3.3 % to 6.7 % (2/30 fully parse-clean samples).

Qualitative inspection of the generated SVG files revealed the remaining failure modes:

- **Hallucinated coordinates**: some path data contains values inconsistent with the declared `viewBox`, though less extreme than in the initial run.
- **Malformed namespace/attribute syntax**: common patterns include `xmlnssvg` (merged tokens) and `viewBox` with repeated values (e.g., `0.0 0.0 24.0 24.0 24.0 24.0`).
- **Invalid colour literals**: hex strings of non-standard length.

The contrast between test perplexity (2.62) and generation quality (6.7 % XML-valid) is a well-known phenomenon in structured-output language modelling. Perplexity measures local token prediction accuracy under teacher forcing; auto-regressive generation compounds errors over hundreds of steps, and a single malformed attribute fails the entire sample.

5 Design Decision Analysis

5.1 BPE Vocabulary Size (4 096)

A smaller vocabulary (e.g., 1 024) would represent SVG numbers as many short tokens, increasing sequence length and memory pressure. A larger vocabulary (e.g., 16 384) would require more embedding parameters than small models can afford and would be harder to train from scratch on ∼100 M tokens. 4 096 balances these trade-offs: SVG keywords and attribute names become single tokens, while numeric literals are represented compactly as 2–4 tokens per number.

5.2 Context Length (512)

As shown in Figure 3, the median SVG in the cleaned corpus requires fewer than 512 tokens. Longer contexts (1 024, 2 048) would incur quadratically more attention memory and render the XL model infeasible on a single GPU with the given batch size.

5.3 Weight Tying (SP Only)

Tying the token embedding matrix to the output projection reduces parameter count and regularises the output distribution. Under µP, the output projection is replaced by `MuReadout` whose per-unit scale must be independent of the embedding, making weight tying incompatible. This explains the slightly higher parameter counts for µP models in Table 5.

5.4 Cosine LR with 200-Step Warmup

The cosine schedule with a short linear warm-up is empirically robust across model sizes in both conditions. The warm-up prevents large gradient updates from corrupting the embeddings in the first few hundred steps. The cosine tail provides implicit LR annealing without requiring a separate scheduling decision.

5.5 98 / 1 / 1 Split by Document

Splitting by document ensures that no sub-sequence of a training SVG appears in validation or test. Token-level splits would create leakage: the model could see the first 400 tokens of an SVG in training and the last 112 in validation, artificially inflating validation performance.

6 Discussion

Why did SP Large and XL diverge?

Under SP, the effective learning rate for a given layer scales with the layer's fan-in at initialisation, which grows with model width. A learning rate optimal for a 1.4 M model is therefore too large for a 88 M model, causing gradient explosion or limit-cycle behaviour. The best-checkpoint losses (1.020 and 1.067 for Large and XL) confirm that both models were initially learning before diverging — consistent with a learning-rate instability, not a fundamental architecture problem. Separately tuning the LR for each size would fix this, but defeats the purpose of a systematic scaling study; µP solves it by construction.

Why does a gap remain between perplexity and generation quality?

Even after switching to the µP-XL model and raising the token budget to 600, XML validity sits at 6.7 % — well below what the perplexity of 2.62 might suggest. Two fundamental factors explain the residual gap.

Perplexity measures local, not global, correctness. Perplexity is computed under teacher forcing: at each step the model sees the true preceding tokens and predicts only the next one. Valid SVG, however, requires *global* constraints: every opened tag must be closed, path command argument counts must be internally consistent, and coordinate values must lie within the declared `viewBox`. These constraints span hundreds of tokens and are invisible to the next-token loss. Auto-regressive generation compounds small local errors, so a single malformed attribute or merged namespace token (`xmlnssvg`) is enough to fail the entire sample.

One epoch is insufficient for structural internalisation. At one epoch over ∼100 M tokens, the model has learned strong local token co-occurrence patterns but has not seen enough examples

to fully memorise the global grammar of SVG. Multi-epoch training or a much larger dataset would be needed to close this gap.

What does the scaling exponent mean?

The µP scaling exponent $\alpha = 1.01$ is substantially higher than the 0.07–0.08 reported for large language models on internet text [Kaplan et al., 2020]. Two factors likely explain this. First, our models are trained for only one epoch, placing us firmly in the compute-limited regime where each additional parameter helps more than an additional pass over the data. Second, SVG is a far more structured domain than natural language, so small models are disproportionately penalised by their inability to track long-range structural dependencies such as matching open/close tags across hundreds of tokens. Both effects steepen the observed loss-vs-parameters curve.

The near-convergence of the µP extrapolation to the fitted floor ($L_{\text{ext}} \approx c = 0.486$) suggests that the current dataset of ∼100 M tokens is the binding constraint for models larger than ∼36 M parameters. Scaling the data alongside model size (as prescribed by Hoffmann et al. [2022]) would be the most impactful next step.

7 Conclusion

We have demonstrated that neural scaling laws apply to SVG language models, but only when the parameterisation is stable across scales. Standard parameterisation (SP) fails for models larger than ∼12 M parameters when the learning rate is fixed from a Tiny-model sweep, producing diverged runs that corrupt the power-law fit. Maximal Update Parameterisation (µP) resolves this: all five µP models converge stably, the fitted scaling law is well-conditioned ($\alpha = 1.01$, CI ± 0.040 at the extrapolation point), and the loss floor $c \approx 0.49$ suggests a fundamental limit of SVG predictability at this data scale.

The µP XL model achieves a test perplexity of 2.62 after extended fine-tuning. Switching to this model and increasing the generation token budget from 400 to 600 improved structural validity from 3.3 % to 23.3 % and XML validity from 3.3 % to 6.7 %, demonstrating that parameterisation choice and generation budget both matter significantly beyond the training loss. Future directions include multi-epoch training, Chinchilla-optimal data scaling, grammar-constrained decoding, and evaluation on downstream tasks such as icon retrieval or text-conditioned SVG generation.

References

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *NeurIPS*, 2022.

Hoffmann, J., Borgeaud, S., Mensch, A., Buchatskaya, E., Cai, T., Rutherford, E., et al. Training compute-optimal large language models. In *NeurIPS*, 2022.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., et al. Scaling laws for neural language models. *arXiv:2001.08361*, 2020.

Karpathy, A. nanoGPT. https://github.com/karpathy/nanoGPT, 2022.

Rodriguez, J. M., Lezama, J., Tsai, Y.-H. H., Milanfar, P., and Schmid, C. StarVector: Generating SVG code from images and text. *arXiv:2312.11556*, 2023.

Yang, G., Hu, E. J., Babuschkin, I., Sidor, S., Liu, X., Farhi, D., et al. Tensor programs V: Tuning large neural networks via zero-shot hyperparameter transfer. *arXiv:2203.03466*, 2022.